

Ex. No. 7	BUILDING AGGREGATION PIPELINES
Date: 02:03:2026	

Aim:

To perform aggregation pipeline operations for Vehicles and Maintenance collections in MongoDB.

Vehicle Collection:

```
db.vehicles.insertMany([
    { vehicleId: 1, vehicleName: "Honda City", ownerName: "Rahul", serviceType: "Oil Change", serviceCost: 3000 },
    { vehicleId: 2, vehicleName: "Hyundai i20", ownerName: "Arun", serviceType: "Brake Service", serviceCost: 5000 },
    { vehicleId: 3, vehicleName: "Toyota Innova", ownerName: "Karthik", serviceType: "Engine Check", serviceCost: 7000 },
    { vehicleId: 4, vehicleName: "Kia Seltos", ownerName: "Vignesh", serviceType: "Battery Service", serviceCost: 4000 },
    { vehicleId: 5, vehicleName: "Mahindra XUV700", ownerName: "Suresh", serviceType: "Full Service", serviceCost: 9000 }
])
```

Aggregation pipeline queries:

Filter vehicles with service cost greater than 4000

```
>db.vehicles.aggregate([
    { $match: { serviceCost: { $gt: 4000 } } }
])
```

Output:

```
{ vehicleId: 2, vehicleName:
"Hyundai i20", ownerName:
"Arun", serviceType: "Brake
Service", serviceCost: 5000
} {
vehicleId:
3,
vehicleName:
e: "Toyota
Innova",
```

ownerName

: "Karthik",

serviceType

: "Engine

Check",

serviceCost:

7000

}

{ vehicleId: 5, vehicleName:

"Mahindra XUV700", ownerName:

"Suresh", serviceType: "Full

Service", serviceCost: 9000

}

Group vehicles by service type and get average service cost

>db.vehicles.aggregate([

{

\$group: { _id: "\$serviceType",

avgServiceCost: { \$avg: "\$serviceCost"

}, count: { \$sum: 1 } }

}

}

])

Output:

{

_id: "Oil Change",

avgServiceCost: 3000,

count: 1

}

{

_id: "Brake Service",

avgServiceCost: 5000,

count: 1

```

    }
    {
      _id: "Engine Check",
      avgServiceCost: 7000,
      count: 1
    }
  }

```

Project vehicle name and a new field 'serviceLevel'

```

>db.vehicles.aggregate([
  {
    $project: {
      vehicleName: 1,
      serviceCost: 1,
      serviceLevel: {
        $switch: {
          branches: [
            { case: { $gte: ["$serviceCost", 8000] }, then: "Premium" },
            { case: { $gte: ["$serviceCost", 5000] }, then: "Standard" },
            { case: { $gte: ["$serviceCost", 3000] }, then: "Basic" }
          ],
          default: "Low"
        }
      }
    }
  }
])

```

Output:

```

{ vehicleName: "Honda
City", serviceCost: 3000,
serviceLevel: "Basic"
}
{ vehicleName: "Hyundai
i20", serviceCost: 5000,
serviceLevel: "Standard"
}

```

```
{  
  vehicleName: "Mahindra  
  XUV700", serviceCost: 9000,  
  serviceLevel: "Premium"  
}
```

Sort vehicles by service cost descending

```
>db.vehicles.aggregate([  
  { $sort: { serviceCost: -1 } }  
])
```

Output:

```
{ vehicleId: 5, vehicleName:  
"Mahindra XUV700", ownerName:  
"Suresh", serviceType: "Full  
Service", serviceCost: 9000  
} { vehicleId: 3, vehicleName:  
"Toyota Innova", ownerName:  
"Karthik", serviceType:  
"Engine Check", serviceCost:  
7000  
} { vehicleId: 2,  
vehicleName: "Hyundai i20",  
ownerName: "Arun",  
serviceType: "Brake  
Service",  
serviceCost: 5000}
```

Result:

Thus the above queries to perform aggregation pipeline operations for Vehicles and Maintenance collections in MongoDB are executed successfully.